

SEW-Offload: 面向昇腾 NPU 的 MoE Expert Offloading

Static Expert-Slot Windowing for Ascend MoE Offloading

不要把 NPU 当 GPU 用： 重点研究如何利用 Ascend 的差异化特性重塑 MoE expert offloading 的设计空间，并据此重新设计专家权重驻留、搬运、窗口化执行与预取机制。

What is your problem

问题背景

混合专家模型通过为每个 token 仅激活少量专家来扩展模型容量，但专家权重本身仍然给推理阶段的设备显存带来巨大压力。专家卸载因此成为在有限 HBM 条件下部署大规模 MoE 模型的重要路径。然而，现有 MoE offloading 系统大多沿用 GPU 时代的抽象，主要关注哪些专家应常驻显存、缺失专家何时从主机侧加载，以及如何进行专家缓存和预取。

现有方法的 GPU 假设

- **expert caching**: 把 HBM 看作 expert cache，热 expert 常驻，冷 expert 放在 host；
- **host-to-device prefetching**: 根据历史路由或预测提前把可能需要的 expert 搬到设备；
- **CUDA stream overlap**: 依赖动态 kernel、stream 并发和异步拷贝隐藏传输开销。



在 Ascend NPU 上
这些 GPU 时代的抽象
不再完整
↓ 详见下页

核心问题定义： 现有 MoE 卸载多把专家权重当作动态缓存对象，依靠替换、预取和拷贝/计算重叠来减少传输开销。但在 HBM 受限的昇腾 NPU 上，动态装载专家会破坏**稳定权重地址、固定执行窗口、图重放和显式搬运调度**。导致要么专家加载暴露在关键路径上，要么牺牲高效 NPU 执行所需的静态规整性。

为什么 GPU 抽象在 Ascend NPU 上不完整

昇腾 NPU 的三类差异化特性，决定了 GPU 式 offloading 不能直接迁移

1. 静态图执行偏好

- **NPUGraph / ACLGraph capture-replay**: 录制完整执行图后以极低开销重放; 捕获时 tensor 地址、shape、dtype 被冻结, 重放期间**不允许**改变。
- **Static Kernel 编译**: 固定 shape 算子可提前编译为高度优化 binary; shape 变体每增一种就多编译一份。
- **稳定权重地址是关键约束**: 动态装载 expert → 权重地址每轮变化 → graph capture 失效 → 退回 eager 模式; CUDA 上轻量的动态 kernel launch 在 Ascend 上代价高昂。

2. 显式数据搬运编排

- **三个独立搬运单元**: 三套搬运引擎各有专用通路: MTE1 走 L1→L0A/L0B (喂矩阵), MTE2 走全局内存 →L1/UB (从 HBM 取数), MTE3 走 UB→ 全局内存 (写回结果)。
- **搬运与计算靠事件同步锁协调**: 搬运和计算各有独立指令队列, 可并行但需显式插入 SetFlag/WaitFlag 控制依赖——前一步没做完后一步必须等。
- **不编排就变成计算空等**: GPU 的隐式并发在昇腾上不存在; 数据预取、权重加载、矩阵计算、结果写回必须组织成 Ping-Pong 双缓冲流水, 否则搬运耗时直接暴露为矩阵单元的停顿。

3. 异构计算单元分工

- **矩阵计算单元**: 专门针对大规模矩阵乘优化, 适合 MoE MLP 的主计算负载;
- **向量计算单元**: 适合元素级操作, 如激活函数、路由选择、重排与合并;
- **标量单元与指令调度**: 把不同阶段计算分配到合适单元, 避免小专家执行被调度开销淹没。
- **GPU 卸载为何不能直接搬**: GPU 上一个核包揽矩阵乘、激活和搬运, 动态装载 expert 直接跑; 昇腾上这些被拆到三条独立流水线——动态装载让同步时序每轮变化, 无法预编排, 只能串行等。**固定槽位让编排可复用**: 槽位地址和尺寸不变, 只替换专家到槽位的映射即可。

核心洞察: Ascend NPU 不是“慢一点的 GPU”, 而是一台有**静态图执行器、独立搬运引擎和异构计算单元**的加速器。GPU 的动态 cache 抽象忽略这三个维度, 直接移植 MoE offloading 会导致 graph 退化、搬运等待和计算单元错配。SEW-Offload 的设计正是围绕这三个特性展开。

Why it matters?

MoE、Ascend 与 offloading 三个层面的必要性

随着 DeepSeek-V3/R1、Qwen-MoE、Mixtral 等模型普及，MoE 已成为大模型扩展的重要方向。问题不再只是能不能训练更大的模型，而是如何让超大 expert 参数在真实硬件上高效服务。

1. 为什么 MoE 很重要

- **扩展参数规模：**拥有巨大专家参数池，但每个 token 只激活少量 expert；
- **提高推理性价比：**相比同等总参数 dense 模型，用稀疏激活降低单 token 计算量；
- **代表前沿趋势：**DeepSeek、Qwen、Mixtral 表明，大规模 MoE 已成为高能力模型的重要形态。

2. 为什么 Ascend 上重要

- **国产算力主战场：**生产环境依赖 Ascend 910B/910C，MoE 能否跑好直接影响算力可用性；
- **不能只复制 GPU：**Ascend 更强调稳定地址、固定 shape、图重放和显式搬运调度；
- **需要 NPU-native 路线：**把 MoE 适配 Ascend，才能释放 AIC/AIV/MTE 等硬件能力。

3. 为什么 Ascend offload 重要

- **HBM 直接卡部署：**大 MoE expert 权重放不下单卡 HBM，必须有可用卸载路径；
- **GPU-style offload 不够：**expert cache/prefetch 会破坏稳定地址、固定窗口和图重放；
- **必须保住 NPU 效率：**既要省 HBM，又要让权重加载、MTE 和 grouped MLP 规整调度。

核心动机：MoE 决定了前沿模型的扩展方向，Ascend 决定了这些模型在国产算力上的落地能力；而 Ascend expert offloading 决定了在 HBM 受限时，能否把“参数很大但激活稀疏”的 MoE 真正高效部署起来。SEW-Offload 的价值正是在 Ascend 约束下，把动态 expert 工作集转化为可复用、可预取、可流水化的固定 slot 执行窗口。

Why existing work fails?

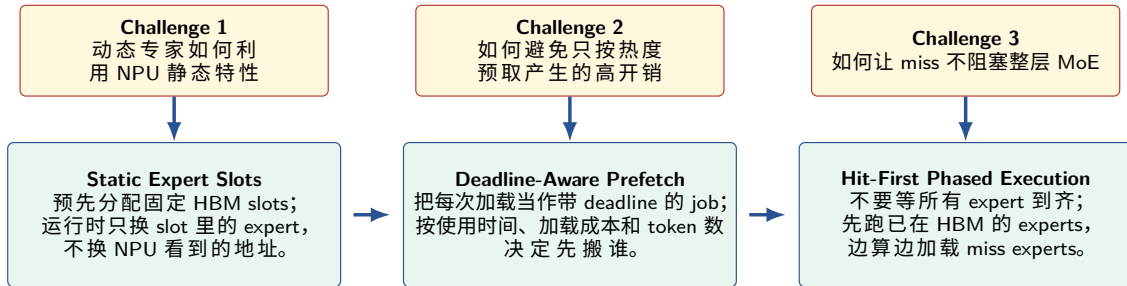
现有工作各自解决了局部问题，但没有同时覆盖 “Ascend + HBM 受限 + expert offload”。

工作类型	代表工作	做法	现有不足
GPU MoE 卸载	MoE-Infinity; Fiddler	将专家作为 GPU 缓存或 CPU-GPU 协同对象，利用激活轨迹、专家热度或执行策略决定缓存替换、预取和 CPU/GPU 分工。	优化目标主要是减少 CPU-GPU 传输并提高 GPU 利用率；默认动态专家对象、动态执行入口和流级重叠可以接受，未处理 Ascend 对稳定权重地址、固定窗口、图重放和 MTE 编排的要求。
Ascend 全量 MoE	vLLM Ascend AscendFusedMoE; npu_grouped_matmul	将路由结果整理成每个专家的 token 计数和 group_list，再调用 Ascend 分组矩阵乘执行 MoE MLP。	默认专家权重已经全量驻留 HBM；没有主机侧专家存储、固定驻留位置、缺失专家预取与替换，也没有针对卸载缺失的分阶段执行。
非 Ascend NPU MoE	NPUMoE	面向 Apple ANE，将稠密和静态计算卸载到 NPU；动态路由、top-k、分散/聚集等不规则部分回退到 CPU/GPU，并用静态分层、分组执行和图常驻降低开销。	目标硬件、内存体系和运行时不同；解决的是 ANE 静态图与动态算子回退问题，不解决 Ascend 上主机到 HBM 的专家权重装载、稳定槽位入口和 MTE/Cube 流水。

研究空白： 缺少面向 Ascend 的 **HBM 受限专家卸载**：不改变路由器、top-k 和 token 语义，同时处理权重驻留、搬运隐藏和稳定执行入口。

What is your key idea?

把 expert offloading 从 “cache hit rate 问题” 改写为 “可静态化、可预取、可分阶段执行的调度问题”。



核心思路：先给 NPU 预留一组固定的专家位置，让执行入口和权重地址保持稳定；再根据每个缺失专家什么时候会被用到、搬运代价有多高、影响多少 token，决定先搬谁；如果仍然来不及搬完，就先执行已经在显存里的专家，同时继续搬运缺失专家，避免整层 MoE 因少数缺失专家而停住。